

Process Operations

中国人民大学信息学院

卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

Outline

- Process id
- Exiting Processes
- Creating Processes
- Reaping Child Processes
- Putting Processes to Sleep
- Loading and Running Programs

Obtaining Process ID's

- **Process ID**
 - **PID**
 - Each process has a unique positive PID
- **Getpid()**
 - Returns the PID of the calling process
- **Getppid()**
 - Returns the PID of its **parent**
 - The process that created the calling process

Obtaining Process ID's

```
#include <unistd.h>
#include <sys/types.h>
pid_t getpid(void) ;
pid_t getppid(void) ;
```

returns: PID of either the caller or the parent

- The `getpid` and `getppid` routines return an integer value of type `pid_t`
- `pid_t`
 - Defined in `types.h` as an `int` on Linux systems

Exit Function

```
#include <stdlib.h>
void exit(int status);
```

this function does not return

- The `exit` function terminates the process with an `exit status` of `status`.

如何创建和初始化进程？

- 建一个空的数据结构，再填充每个域
- 复制（内核**init**进程是所有进程的祖先，**pid=1**）
 - **fork()**
 - 创建新进程（子进程），由父进程复制而来
 - 两个进程都从**fork**返回
 - 父进程返回创建的子进程**PID**
 - 子进程返回**0**
 - **exec(...)**
 - 参数传进来一个**program**，更换当前进程的**code**和**data**，执行传进来的**program**指令

Fork Function

- A parent process
 - creates a new running child process
 - by calling the `fork` function

```
#include <unistd.h>
#include <sys/types.h>
pid_t fork(void);
```

Returns: 0 to child, PID of child to parent, -1 on error

Fork Function

- 新创建的子进程
 - 几乎，但不是完全地，与父进程相同
- 父进程和新创建的子进程有不同的PID

Fork Function

- 子进程
 - 得到一份父进程用户层虚拟机地址空间的完全拷贝
 - including the **text**, **data**, and **bss** segments, **heap**, and user **stack**.
 - 同时也得到父进程已打开的文件描述符（ **file descriptors** ）的完全拷贝
 - 意味着子进程可以直接读，写父进程中已经打开的任何文件

Fork Function

- Called once
 - In the parent process
- Returns twice:
 - In the parent process
 - Return the **PID** of the child
 - In the newly created child process.
 - Return **0**
- The return value provides an unambiguous way
 - whether the program is executing in the parent or the child.

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5     pid_t pid;
6     int x = 1;
7
8     pid = fork();
9     if (pid == 0) { /* child */
10         printf("child : x=%d\n", ++x);
11         exit(0);
12     }
13
14     /* parent */
15     printf("parent: x=%d\n", --x);
16     exit(0);
17 }
```

Fork Function

- Concurrent execution
 - The parent and the child are **separate** processes that run **concurrently**.
 - The instructions in their logical control flows can be **interleaved** by the kernel in an arbitrary way.
 - We can never make assumptions about the interleaving of the instructions in different processes.

Fork Function

- Duplicate but separate address spaces
 - Immediately after the `fork` function returned in each process, the address space of each process is identical.
 - Local variable `x` has a value of 1 in both the parent and the child when the `fork` function returns in line 8.

Fork Function

- Duplicate but separate address spaces
 - The variable `x` has different values in the parent and child when they call their respective `printf` statements. (`++x/--x`)
- Shared files
 - Like `stdout`
 - Communication between child and parent

Fork Function

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {                // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {                    // parent goes down this path (main)
16         printf("hello, I am parent of %d (pid:%d)\n",
17               rc, (int) getpid());
18     }
19     return 0;
20 }
```

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

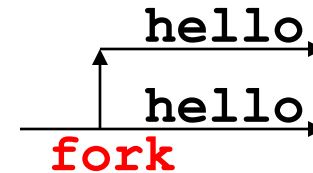
Fork Function

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int
7  main(int argc, char *argv[])
8  {
9      printf("hello world (pid:%d)\n", (int) getpid());
10     int rc = fork();
11     if (rc < 0) {          // fork failed; exit
12         fprintf(stderr, "fork failed\n");
13         exit(1);
14     } else if (rc == 0) { // child (new process)
15         printf("hello, I am child (pid:%d)\n", (int) getpid());
16     } else {                // parent goes down this path (main)
17         int wc = wait(NULL);
18         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
19                rc, wc, (int) getpid());
20     }
21     return 0;
22 }
```

prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5     Fork();
6     printf("hello!\n");
7     exit(0);
8 }
```

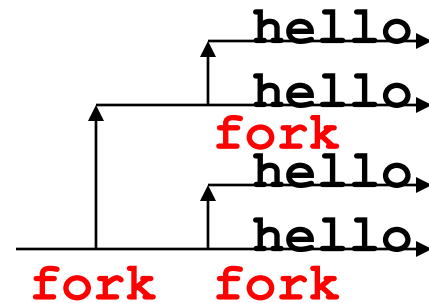


(a) Calls fork **once**.

(b) Prints **two** output lines.

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5     Fork();
6     Fork();
7     printf("hello!\n");
8     exit(0);
9 }
```

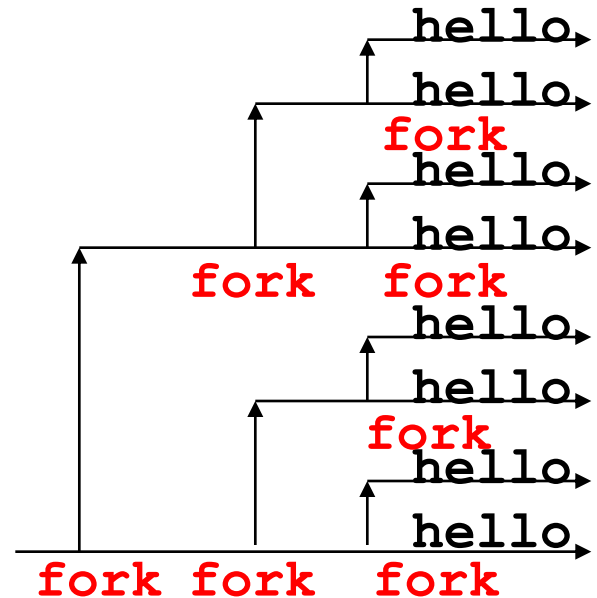


(c) Calls fork **twice**.

(d) Prints **four** output lines.

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5     Fork();
6     Fork();
7     Fork();
8     printf("hello!\n");
9     exit(0);
10 }
```



(e) Calls fork **three** times. (f) Prints **eight** output lines.

课堂练习

code/ecf/forkprob0.c

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int x = 1;
6
7      if (Fork() == 0)
8          printf("printf1: x=%d\n", ++x);
9      printf("printf2: x=%d\n", --x);
10     exit(0);
11 }
```

code/ecf/forkprob0.c

- A. 子进程的输出是什么？
- B. 父进程的输出是什么？
- C. 共多少种输出可能？（假设每行一定完整输出）

课堂练习

这个程序会输出多少个“hello”输出行？

code/ecf/forkprobl.c

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int i;
6
7      for (i = 0; i < 2; i++)
8          Fork();
9      printf("hello\n");
10     exit(0);
11 }
```

code/ecf/forkprobl.c

课堂练习

这个程序会输出多少个“hello”输出行？

code/ecf/forkprob4.c

```
1  #include "csapp.h"
2
3  void doit()
4  {
5      Fork();
6      Fork();
7      printf("hello\n");
8      return;
9  }
10
11 int main()
12 {
13     doit();
14     printf("hello\n");
15     exit(0);
16 }
```

code/ecf/forkprob4.c

Zombie

- **Kernel**并不会在进程终止后立刻将其清理掉
- 已终止的进程一般会保持在**terminated**状态，直到被其**parent**收割（**reaped**）

Zombie

- When the parent **reaps** the terminated child
 - 内核首先把child的exit status发送给parent
 - 然后抛弃已经终止的进程
- A terminated process that has not yet been reaped is called a **zombie**.

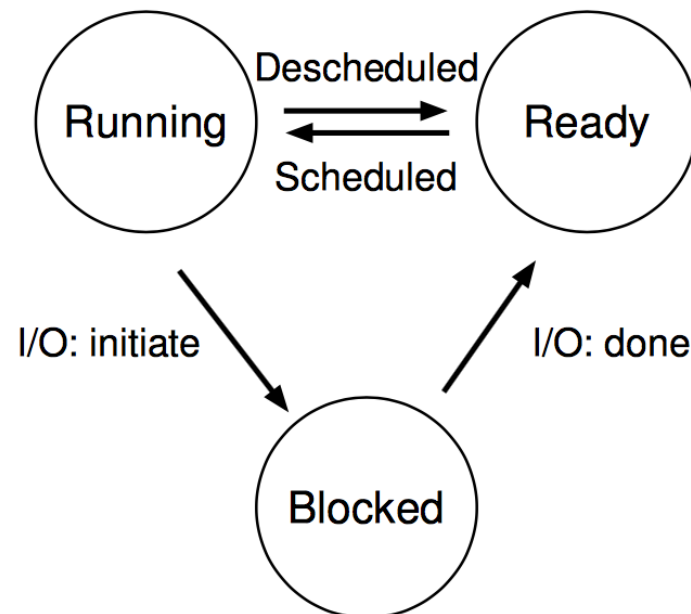
Zombie

- If the parent process
 - terminates without reaping its zombie children,
 - the kernel arranges for the **init** process to reap them.
- The **init** process
 - has a PID of 1
 - and is created by the kernel during system initialization.

Zombie

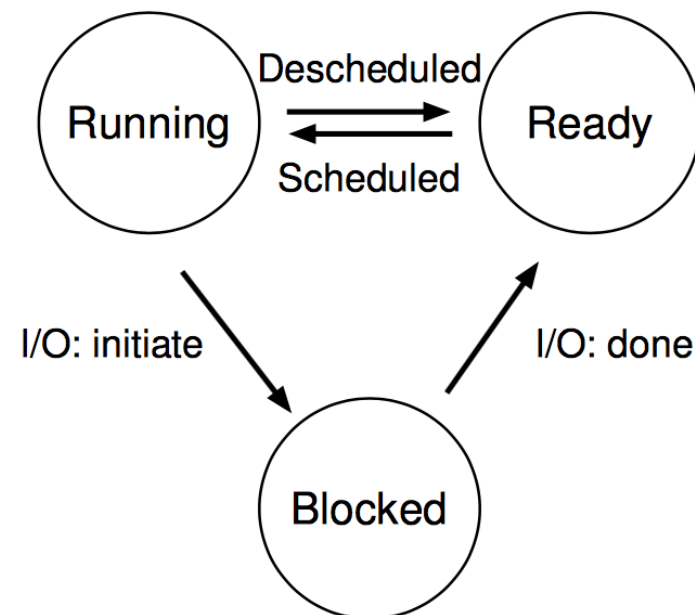
- 长期运行的程序（例如**shells**或**servers**），应该总是收割他们的**zombie children**
- 即使**zombie**不在运行，没有消耗**CPU**资源，他们也消耗了系统的内存资源

Review of Process Status



- Running
- Ready（就绪）
- **Blocked/Stopped（停止/挂起/阻塞：不会被调度）**
 - 进行I/O访问、调用sleep、访问锁等，可能进入阻塞状态
 - 收到SIGSTOP、SIGTSTP、SIGTTIN或SIGTTOU信号时，进程被停止
 - 收到SIGCONT，进入运行状态
- Terminated（终止）

Review of Process Status



- Running
- Ready（就绪）
- Blocked/Stopped（停止/挂起/阻塞：不会被调度）
- Terminated（终止）三个原因：
 - 收到一个信号，该信号的默认行为是终止进程；
 - 从主程序返回（从main函数返回一个整型）；
 - 调用exit函数（记录status）

课堂练习

- 进程阻塞的原因不包括_____。
 - A. 时间片切换
 - B. 等待I/O
 - C. 进程sleep
 - D. 等待解锁

Wait_pid Function

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

```
pid_t wait(int *status);
```

Returns: PID of child if OK, 0 (if WNOHANG) or -1 on error

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- `pid > 0`
 - The wait set is the **singleton** child process
 - whose process ID is equal to **pid**
- `pid = -1`
 - The wait set consists of **all** of the parent's child processes

Wait_pid Function

- The `waitpid` function
 - If the calling process has **no** children
 - returns -1
 - sets `errno` to **ECHILD**
 - If interrupted by a **signal**
 - returns -1
 - sets `errno` to **EINTR**

>>

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options = 0
 - waitpid函数的调用进程进入挂起状态，直到它的wait-set中的一个child process终止了；
 - 如果wait-set中的一个进程在进行waitpid调用的时候已经终止了，那么waitpid立刻返回；
 - Waitpid返回导致waitpid函数返回的、已经终止的child process的PID
 - 已中止的child process被从系统中删除，释放其资源

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options
 - **WNOHAN**: 如果wait-set中的任何子进程都还没有终止，那么立即返回，返回值为0
 - 如果在等待子进程终止的过程的同时，还希望做些有用的工作，这个选项会有用。

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options
 - **WUNTRACED**: 将调用waitpid的进程挂起，直到wait-set中的一个子进程已经变成已终止或者被停止。返回已终止或已停止的子进程PID。

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- options
 - WCONTINUED: 将调用waitpid的进程挂起，直到wait-set中的一个正在运行的子进程终止，或者等待集合中一个被停止的进程收到SIGCONT信号重新开始。

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Checking the exit status of a reaped child
 - The **status** argument is non-NULL
 - `waitpid` encodes status information about the child
 - that caused the return in the `status` argument.
 - The `wait.h` include file defines several **macros**
 - for interpreting the `status` argument

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFEXITED(status)**
 - Returns true if the child terminated normally
 - via a call to `exit` or a return.
- **WEXITSTATUS(status)**
 - Returns the **exit status** of a normally terminated child.
 - This status is only defined if **WIFEXITED** returned true.

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFSIGNALED(status)**
 - Returns true if the child process terminated because of a **signal** that was not caught
- **WTERMSIG(status)**
 - Returns the **number of the signal** that caused the child process to terminate.
 - This status is only defined if **WIFSIGNALED** returned true.

Wait_pid Function

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- **WIFSTOPPED(status)**
 - Returns true if the child that caused the return is currently **stopped**.
- **WSTOPSIG(status)**
 - Returns the **number of the signal** that caused the child to stop.
 - This status is only defined if **WIFSTOPPED** returned true.

wait

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *status, int options);
pid_t wait(int *status);
```

Returns: PID of child if OK, 0 (if WNOHANG) or -1 on error

- `wait(&status)`是`waitpid`的简化版，相当于
 - `waitpid(-1, &status, 0);`

Putting Process to Sleep

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int secs);
```

Returns: seconds left to sleep

```
int pause(void);
```

Always returns -1

- **sleep**

- 挂起一个进程一段时间;
- 如果请求的时间量已经到了, 则返回0;
- 否则返回剩下还要休眠的秒数
 - 如果**sleep**函数被一个信号中断而过早返回时, 会没有休眠足够的描述而提前返回。

Putting Process to Sleep

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int secs);
```

Returns: seconds left to sleep

```
int pause(void);
```

Always returns -1

- **pause**
 - 让调用函数休眠，直到该进程收到一个信号

Loading and Running

```
#include <unistd.h>
```

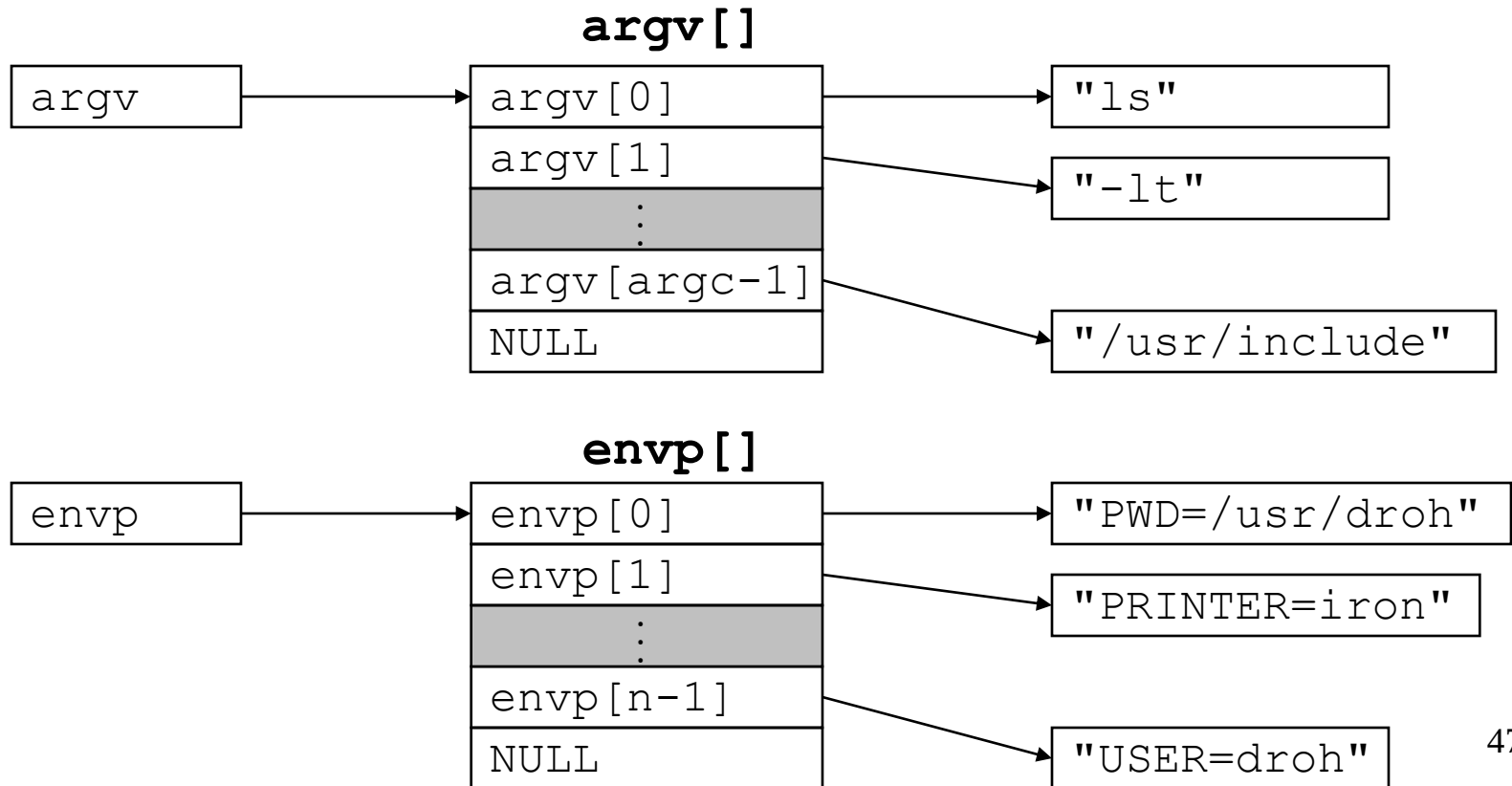
```
int execve(const char *filename, const char *argv[],  
           const char *envp[]);
```

does not return if OK, returns -1 on error

- Loads and runs
 - the executable object file `filename`
 - with the argument list `argv`
 - the environment variable list `envp`.
 - returns to the calling program only if there is an error
 - such as not being able to find `filename`.
- The `Execve` is called once and **never** returns

Loading and Running

```
int execve(const char *filename, const char *argv[],  
           const char *envp[]);
```

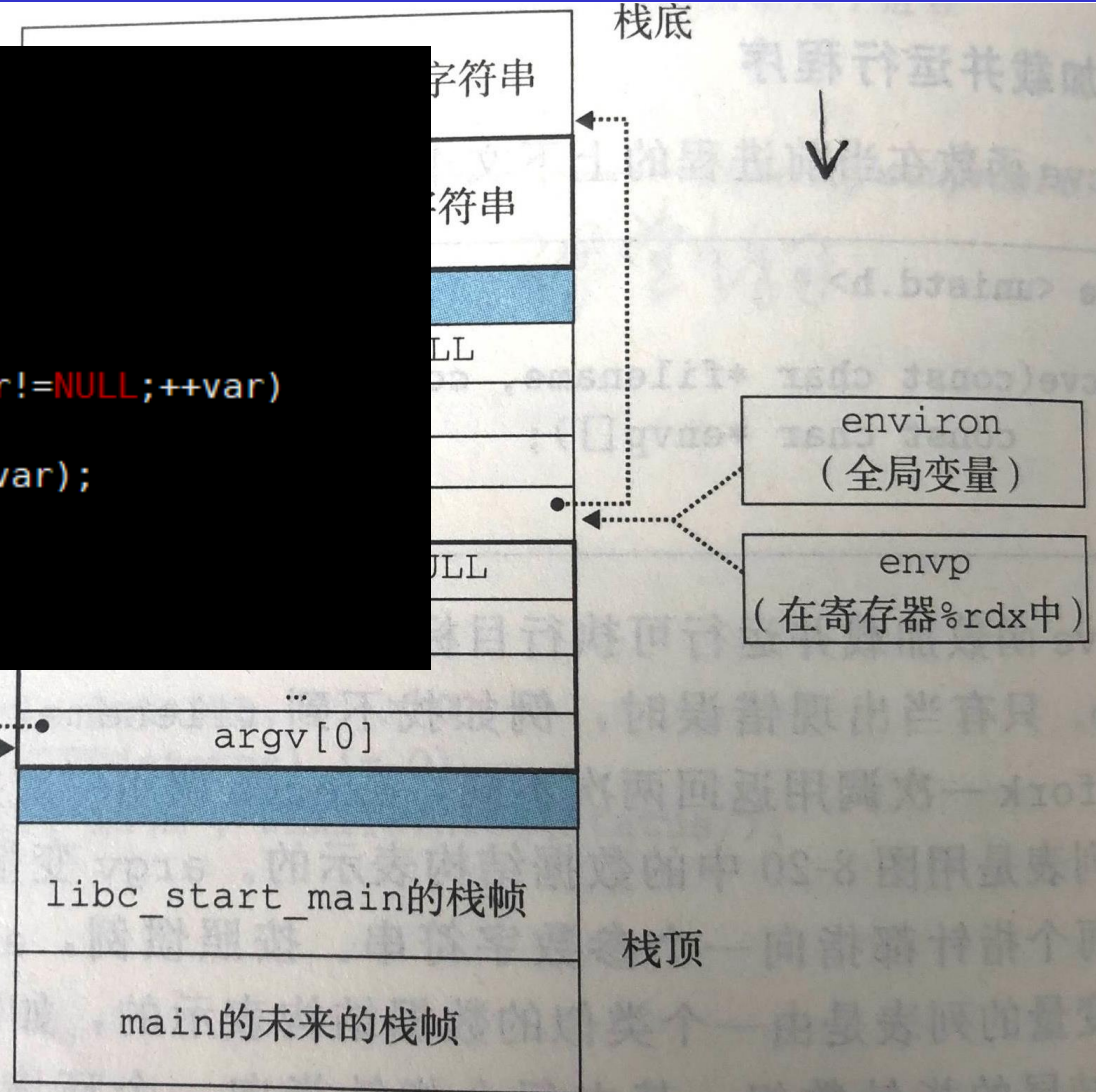


The user stack when a new program starts

```
1 #include <stdio.h>
2
3 extern char **environ;
4
5 int main(void)
6 {
7     char **var;
8     for(var=environ;*var!=NULL;++var)
9     {
10         printf("%s\n",*var);
11     }
12     return 0;
13 }
14 }
```

argv
(在寄存器%rsi中)

argc
(在寄存器%rdi中)



Loading and Running

```
#include <unistd.h>
```

```
char *getenv(const char *name) ;
```

Returns: ptr to name if exists, NULL if no match.

```
int setenv(const char *name, const char *newvalue,  
           int overwrite) ;
```

Returns: 0 on success, -1 on error.

```
void unsetenv(const char *name) ;
```

Returns: nothing.

Exec

```
1  #include <stdio.h>                                prompt> ./p3
2  #include <stdlib.h>                                hello world (pid:29383)
3  #include <unistd.h>                                hello, I am child (pid:29384)
4  #include <string.h>                                29      107      1030 p3.c
5  #include <sys/wait.h>                              hello, I am parent of 29384 (wc:29384) (pid:29383)
6                                          prompt>
7  int
8  main(int argc, char *argv[])
9  {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) { // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc"); // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL; // marks end of array
21         execvp(myargs[0], myargs); // runs word count
22         printf("this shouldn't print out");
23     } else { // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26             rc, wc, (int) getpid());
27     }
28     return 0;
29 }
```

wc统计文件的行数、字数、字节数
一个字被定义为由空白、跳格或换行
字符分隔的字符串

Unix Shell

- An interactive application-level program that
 - runs other program on behalf of the user
- Variants: sh, csh, tcsh, ksh, bash
- Performs a sequence of read/evaluate steps and terminate
 - Read: reads a command line from the user
 - Evaluate: parses the command line and runs programs on behalf of the user

Parsing command line

- Parse the space separated command-line arguments
- Builds the `argv` vector
 - which will eventually be passed to `execve`

Foreground and Background

- If the last argument is a "&" character
 - parseline returns 1
 - indicating the program should be executed in the background
(the shell returns to the top of the loop and waits for the next command)
- Otherwise
 - parseline returns 0
 - indicating the program should be run in the foreground
(the shell waits for it to complete)